

KARBOWANEC

Hard Fork Specification

Confidential Transactions

Version 2.0 — Draft
Karbowanec Development

This document specifies the KarboCT confidential transaction protocol for the Karbowanec network.

CT transfers are mandatory for all non-coinbase transactions after activation at `UPGRADE_HEIGHT_V6`.

Each output commits to one of 64 canonical denominations using a Pedersen commitment proved by a

Groth-Kohlweiss one-out-of-many membership proof. Coinbase remains transparent. No redenomination

is introduced: 1 KRB = 10^{12} atomic units throughout.

Item	Value
GK proof size	1,376 bytes (24 points + 19 scalars, $N=64$, $n=6$)
$Q[0..5]$	All real blinded coefficient commitments; $Q[0]$ is not identity
Fiat-Shamir order	<code>domain D[*] I[*] A[*] B[*] Q[*] tx_signing_hash</code>
Ring size (consensus)	4..16; <code>ringSize=1</code> for V5+ coinbase carve-out only
Wallet default ring	16 for normal CT spends
Min denomination	10^{10} atomic units = 0.01 KRB
Redenomination	None. Precision unchanged: 1 KRB = 10^{12} atomic units
Activation	<code>CT_FORK_HEIGHT = UPGRADE_HEIGHT_V6</code>

Contents

1. Overview
2. Design goals and non-goals
3. Notation
4. Denomination set
5. Pedersen commitments
6. ECDH amount masking and audit visibility
7. Groth-Kohlweiss denomination membership proof
8. Transaction balance equation and kernel
9. MLSAG ring signatures and ring-size policy
10. Subgroup and point validation
11. Transaction structure
12. Transaction validation procedure
13. Fee policy
14. Wallet construction policy
15. Coinbase treatment
16. Activation and compatibility
17. Security properties and known limitations
18. Transaction size estimates
19. Implementation plan
20. Testing priorities

1. Overview

This document specifies the activation of confidential transactions on Karbo, preserving the existing 12-decimal atomic precision ($1 \text{ KRB} = 10^{12}$ atomic units) and CryptoNote-style stealth-address model. The fork introduces a second transaction version for non-coinbase transfers. Coinbase remains transparent because the block reward and collected fees are public consensus data.

Area	Rule
Transaction version	TRANSACTION_VERSION_CT = 2 for post-activation non-coinbase transfers.
Activation	CT_FORK_HEIGHT = UPGRADE_HEIGHT_V6.
Amounts	Hidden inside Pedersen commitments and ECDH-masked 8-byte fields.
Output validity	Each output proves membership in the 64-entry canonical denomination table.
Input authorization	Two-row MLSAG over one-time key and commitment-difference rows.
Balance	Pseudo-output commitments plus fee and kernel signature prove no inflation.
Coinbase	Transparent version-1 coinbase remains unchanged at all heights.

1.1 What changes

- Beginning at activation, regular transfers are CT transactions. Legacy transparent non-coinbase transactions are rejected after the activation height.
- CT output amounts are hidden by Pedersen commitments $C = v \cdot H + r_{1tt} \cdot G$ and masked amount fields.
- Each CT output carries a Groth-Kohlweiss one-out-of-many proof that the committed value is one of the 64 canonical denominations.
- Each CT input is a ConfidentialInput carrying a ring, ring commitments, a pseudo-output commitment, and a key image.
- The transaction has a kernel excess commitment and Schnorr signature proving knowledge of the balancing blinding excess.

1.2 What does not change

- 1 KRB remains 10^{12} atomic units. There is no redenomination and no one-time accumulator conversion.
- Emission accounting, alreadyGeneratedCoins, the money supply constant, block headers, and mining algorithm remain unchanged.
- Coinbase remains transparent and version 1 at all heights.
- Standard CryptoNote one-time stealth address derivation remains unchanged.
- The existing deterministic tx secret-key derivation is preserved for compatibility.

2. Design goals and non-goals

Design intent

KarboCT is deliberately a practical middle ground: better trading and transfer privacy than transparent CryptoNote amounts, with simpler math and implementation scope than arbitrary-amount Bulletproof-style CT.

2.1 Goals

- Hide exact transfer amounts from public observers.
- Preserve CryptoNote sender privacy through rings and stealth addresses.
- Avoid confidential dust by using a minimum CT denomination of 0.01 KRB.
- Enable exchange and business audit flows by allowing a view-key holder to reconstruct outgoing transaction keys.
- Keep the protocol implementable with existing Ed25519, Keccak, CryptoNote hash-to-scalar, and MLSAG-style primitives.

2.2 Non-goals

- KarboCT does not provide arbitrary hidden amounts. It hides one of 64 canonical denominations per output.
- KarboCT does not hide asset type or introduce confidential assets.
- KarboCT does not make coinbase private.
- KarboCT does not eliminate every amount-shape leak; output count can still correlate with payment magnitude.

3. Notation

Symbol	Meaning
G	Ed25519 base point in the prime-order subgroup.
H	Independent amount generator: $H = 8 \cdot \text{elligator}(\text{Keccak}(\text{"CN-amount-generator"}))$.
q or l	Ed25519 prime subgroup order.
Hs(.)	Hash-to-scalar using Keccak followed by scalar reduction.
v	Committed denomination value in atomic units.
r_out	Pedersen output blinding factor (random scalar).
C	Output commitment: $C = v \cdot H + r_{\text{out}} \cdot G$.
V[0..63]	Canonical denomination table, 64 entries.
N	GK proof list size, $N = 64$.
n	GK bit width, $n = \log_2(N) = 6$.
a_s	Sender/auditor view secret used for deterministic tx_key derivation.
A_rec	Recipient view public key used for ECDH derivation.
tx_key	Per-transaction secret scalar; published as $R = \text{tx_key} \cdot G$ in tx_extra.
R	Transaction public key in tx_extra.
inputs_hash	Canonical hash of the fully populated input vector.
tx_signing_hash	<code>getObjectHash(TransactionPrefix)</code> .

Notation note

a_s denotes the sender/auditor view secret used for deterministic tx_key reconstruction.
A_rec denotes the recipient view public key used for ECDH shared-secret derivation.
The two roles are distinct and must not be conflated.

4. Denomination set

The denomination model is the core simplification of KarboCT. A CT output does not prove an arbitrary amount is within a range. It proves the committed amount is one member of a fixed public table.

```
MIN_CT_DENOMINATION =  $10^{10}$  atomic units = 0.01 KRB
```

4.1 Table

The 64-entry set is $d \cdot 10^k$ for d in $\{1..9\}$, k in $\{10..16\}$, plus the cap entry 10^{17} . Values are ordered ascending.

Index range	Atomic units	KRB range
0..8	$1 \cdot 10^{10} .. 9 \cdot 10^{10}$	0.01 .. 0.09 KRB
9..17	$1 \cdot 10^{11} .. 9 \cdot 10^{11}$	0.1 .. 0.9 KRB
18..26	$1 \cdot 10^{12} .. 9 \cdot 10^{12}$	1 .. 9 KRB
27..35	$1 \cdot 10^{13} .. 9 \cdot 10^{13}$	10 .. 90 KRB
36..44	$1 \cdot 10^{14} .. 9 \cdot 10^{14}$	100 .. 900 KRB
45..53	$1 \cdot 10^{15} .. 9 \cdot 10^{15}$	1,000 .. 9,000 KRB
54..62	$1 \cdot 10^{16} .. 9 \cdot 10^{16}$	10,000 .. 90,000 KRB
63	$1 \cdot 10^{17}$	100,000 KRB (cap)

4.2 Decomposition policy

Consensus verifies only that every output is one allowed denomination. It cannot know which destination or change output corresponds to a user-level payment amount. Decomposition is therefore a wallet policy, not a direct consensus rule.

Official wallet decomposition policy

1. Reject destination amounts that are not non-zero multiples of MIN_CT_DENOMINATION.
2. Use greedy canonical decomposition for payment outputs.
3. Use the same canonical logic for change outputs when change is emitted.
4. Fold sub-floor residue into fee; never create CT dust outputs.

Requirement level	Rule
Consensus MUST	Each CT output proof must verify against $V[0..63]$.
Official wallet MUST	Use greedy canonical decomposition to avoid fingerprinting and output bloat.
Non-standard wallet	May create valid but distinguishable decompositions; discouraged.

5. Pedersen commitments

KarboCT uses additive notation over the Ed25519 prime-order subgroup.

$$C = v \cdot H + r_{\text{out}} \cdot G$$

where:

v = one allowed denomination value (from $V[0..63]$)
 r_{out} = uniformly random scalar modulo q
 H = independent amount generator

5.1 Generator H

```
H = ge_mul8( elligator2( Keccak("CN-amount-generator") ) )
```

The cofactor clear places H in the prime-order subgroup. Implementations should compute or verify H once at process startup and use the same canonical serialized generator everywhere.

6. ECDH amount masking and audit visibility

The amount mask and output blinding are derived from the recipient ECDH shared secret. For compatibility with the current implementation and audit design, tx_key remains deterministically derived from the sender/auditor view secret and inputs_hash.

Compatibility-preserved tx key derivation

```
tx_key = Hs( a_s || inputs_hash )  
R      = tx_key * G
```

This derivation is intentionally preserved. It gives a_s-holders full bidirectional visibility over outgoing transactions without exposing spend authority.

Compatibility note

Do not add a domain separator to tx_key derivation unless the chain and wallet compatibility impact is explicitly planned. Existing outgoing audit reconstruction depends on the exact derivation.

6.1 Sender construction

```
shared      = derive_key(A_rec, tx_key)           // ECDH with  
recipient pubkey  
mask_v      = first_8_bytes( Hs(shared || 0x00) ) // 8-byte amount  
mask  
masked_amount = uint64_le(v) XOR mask_v          // stored on-chain  
r_out       = Hs("blinding" || shared || output_index) // per-output  
blinding factor  
C           = v.H + r_out.G                      // Pedersen commitment
```

6.2 Recipient scanning

```
shared      = derive_key(R, recipient_view_secret) // ECDH with tx  
pubkey  
mask_v      = first_8_bytes( Hs(shared || 0x00) )  
v           = uint64_le( masked_amount XOR mask_v )  
r_out       = Hs("blinding" || shared || output_index )  
if C != v.H + r_out.G: skip output              // not addressed to  
this wallet
```

A failed commitment check is not a transaction-validity error. It simply means the output is not addressed to this wallet.

6.3 Outgoing audit property

Given the sender/auditor view secret a_s and the public input vector, an auditor can recompute inputs_hash, reconstruct tx_key, derive R, and recover the shared secrets required to identify outgoing outputs and values. This gives the view key bidirectional visibility without exposing spend authority.

7. Groth-Kohlweiss denomination membership proof

Normative statement

KarboCT uses the standard Groth-Kohlweiss one-out-of-many relation specialized to a 64-element denomination table. The proof is not a Karbo-specific construction. I/A/B are the mandatory binaryity/index-binding commitments of the standard GK proof. Q[0..5] are real blinded lower-degree coefficient commitments; Q[0] is never identity.

7.1 Statement

For each output commitment C, the verifier derives a public list of 64 points:

$$D[k] = C - V[k] \cdot H, \quad \text{for } k = 0..63$$

If the committed value is V[l], then:

$$\begin{aligned} C &= V[l] \cdot H + r_{\text{out}} \cdot G \\ D[l] &= C - V[l] \cdot H = r_{\text{out}} \cdot G \end{aligned}$$

The prover proves in zero knowledge that one element of D[0..63] has known discrete log with respect to G. Equivalently, the prover knows l in [0,63] and r_out such that C = V[l]·H + r_out·G.

7.2 Proof size

Component	Count	Bytes
I[j]	6 points	192
A[j]	6 points	192
B[j]	6 points	192
Q[m]	6 points	192
z[j]	6 scalars	192
za[j]	6 scalars	192
zb[j]	6 scalars	192
f	1 scalar	32
Total	24 points + 19 scalars	1,376

$$\begin{aligned} N &= 64, \quad n = \log_2(N) = 6 \\ \text{proof size} &= 4 \cdot n \text{ points} + (3 \cdot n + 1) \text{ scalars} \\ &= 24 \cdot 32 + 19 \cdot 32 = 1,376 \text{ bytes} \end{aligned}$$

7.3 Bit decomposition and commitments

Let l_j be the six bits of the secret denomination index l . For each bit position j , the prover chooses fresh random scalars r_j , a_j , s_j , and t_j and computes:

```
I[j] = r_j · G + l_j · H      // index commitment
A[j] = s_j · G + a_j · H      // randomness commitment
B[j] = t_j · G + (l_j · a_j) · H // binaryity commitment
```

These commitments are the mandatory binaryity/index-binding part of the GK proof. A shorter form that omits this part is incomplete and must never be accepted on-chain.

7.4 Selector polynomials

For every public index k with bits k_j , define a degree-6 selector polynomial:

```
f_j(1, x) = l_j · x + a_j
f_j(0, x) = (1 - l_j) · x - a_j
p_k(x) = ∏_{j=0..5} f_j(k_j, x)
```

For $k = l$, every factor has leading coefficient 1, so p_l has degree 6. For $k \neq l$, at least one factor is constant-only, so the leading coefficient vanishes.

7.5 $Q[m]$ commitments

Important

$Q[0]$ is a real blinded coefficient commitment. Do not special-case $Q[0]$ as identity. This is normative. Any implementation that sets $Q[0] = \text{identity}$ produces invalid proofs.

Expand $p_k(x) = \sum_{m=0..6} c[k,m] \cdot x^m$. For $m = 0..5$, the prover samples $\rho[m]$ and commits:

```
for m in 0..5:
    Q[m] = rho[m] · G + ∑_{k=0..63} c[k,m] · D[k]
```

There is no serialized $Q[6]$. The degree-6 coefficient is contributed only by the secret index and equals $D[l] = r_{\text{out}} \cdot G$. It is absorbed into the scalar response f :

```
f = r_out · x^6 - ∑_{m=0..5} rho[m] · x^m
```

7.6 Fiat-Shamir transcript

The normative challenge transcript must be identical in prover, verifier, tests, and documentation:

```
x = Hs(
    "GK-KarboCT-v2" ||
    D[0..63]          ||
    I[0..5]           ||
```

```

    A[0..5]          ||
    B[0..5]          ||
    Q[0..5]          ||
    tx_signing_hash
)

```

Implementation note

The normative serialized order is: domain, D, I, A, B, Q, then tx_signing_hash.
Update source comments if they mention tx_hash before D[*] or show a different order.

7.7 Responses

```

for j in 0..5:
    z[j] = l_j · x + a_j
    za[j] = r_j · x + s_j
    zb[j] = r_j · (x - z[j]) + t_j
f = r_out · x6 - Σ_{m=0..5} rho[m] · xm

```

7.8 Verifier

1. Check all scalar encodings: z, za, zb, f in [0, q).
2. Decode and prime-order subgroup-check all proof points: I, A, B, Q.
3. Recompute $D[k] = C - V[k] \cdot H$ and subgroup-check every $D[k]$.
4. Recompute x using the normative Fiat-Shamir transcript above.
5. For each j in 0..5, verify both checks:

```

x · I[j] + A[j]      == za[j] · G + z[j] · H    // index binding
(x - z[j]) · I[j] + B[j] == zb[j] · G          // binaryity

```

6. Compute $p_k(x) = \prod_j (\text{bit}_j(k) ? z[j] : x - z[j])$.
7. Verify the polynomial evaluation check:

```

Σ_k p_k(x) · D[k] == f · G + Σ_{m=0..5} xm · Q[m]

```

8. Transaction balance equation and kernel

Each input carries a pseudo-output commitment:

$$c'_i = v_i \cdot H + r'_i \cdot G$$

The two-row MLSAG proves the signer knows the spend key and the opening difference between the real ring commitment and the pseudo-output commitment.

8.1 Balance equation

$$\sum_i c'_i - \sum_o c_{out_o} - fee \cdot H = excess \cdot G$$

Honest construction cancels all H components because input values equal output values plus fee. The remaining G component is the blinding excess.

8.2 Kernel signature

```
excess_commitment = excess · G  
kernel_sig        = Schnorr_Sign(excess, tx_signing_hash)
```

The verifier recomputes the balance equation and verifies the Schnorr signature under `excess_commitment`. A valid signature proves knowledge of the discrete log of the residual point with respect to G, which is possible only when the H component cancels.

9. MLSAG ring signatures and ring-size policy

Row	Public element at ring index k	True secret at index l
0	P_k, the one-time public key	x_l, the spend private key
1	D_k = C_k - C'_i	r_real - r'_i

Key image: $I = x_l * H_p(P_l)$

9.1 Consensus ring-size constraints

```
CT_MIN_RING_SIZE = 4
CT_MAX_RING_SIZE = 16

ringSize == 1 permitted only for the V5+ coinbase carve-out (see §15.1)
otherwise: 4 <= ringSize <= 16
```

9.2 Wallet default policy

Recommended default

Official wallets SHOULD use ring size 16 for normal CT spends. This improves both decoy count and transaction uniformity. Smaller ring sizes should be reserved for compatibility, low-liquidity edge cases, or controlled internal operations.

Ring size	Approx. bytes/input	Naive real-input guess
4	~636 B	25%
8	~1,180 B	12.5%
16	~2,232 B	6.25%

The privacy benefit of ring size 16 is not only more decoys. It also avoids wallet fingerprinting from variable ring-size choices. For many-input consolidation, a higher voluntary fee should be encouraged through the wallet fee-priority UI.

10. Subgroup and point validation

KarboCT validation uses strict prime-order subgroup checks, not merely cofactor clearing or weak small-order rejection.

```
point_valid(P):
  require P decodes to a valid curve point
  require P is not identity
  require l·P == identity
  return true
```

Point	Where checked	Risk if omitted
Output commitment C	All CT outputs	Commitment malleability or invalid proof statements
targetKey	All CT outputs	Address malleability
Pseudo commitment C'	All CT inputs	Balance bypass
Key image I	All CT inputs	Double-spend or invalid image domain
Excess commitment	Kernel	Forged kernel signature
GK I/A/B/Q	All GK proofs	Membership proof forgery
Derived D[k]	GK verifier	Small-subgroup confusion
MLSAG ring keys	Input ring verification	Ring-signature forgery

11. Transaction structure

CT transactions are represented as a prefix plus body. The prefix is signed and used by the GK transcript, MLSAG signatures, and kernel signature. The canonical full transaction hash still includes the body.

```
TransactionPrefix {
    uint8          version;
    uint64         unlockTime;
    TransactionInput[] inputs;
    TransactionOutput[] outputs;
    bytes          extra;
    uint64         fee;          // CT only
}

Transaction : TransactionPrefix {
    Signature[][]  signatures;    // legacy v1 only
    CTInputSignature[] ctSignatures; // CT v2 only
    CTOutputProof[] ctProofs;     // CT v2 only
    TransactionKernel kernel;     // CT v2 only
}
```

11.1 Confidential input and output

```
ConfidentialInput {
    uint64          ringAmount;
    uint32[]        ringOutputIndexes;
    PublicKey[]     ringPubkeys;
    EllipticCurvePoint[] ringCommitments;
    EllipticCurvePoint pseudoCommitment;
    KeyImage        keyImage;
}

ConfidentialOutput {
    PublicKey        targetKey;
    EllipticCurvePoint commitment;
    uint8[8]        maskedAmount;
}
```

11.2 GK proof and kernel body

```
CTOutputProof {
    EllipticCurvePoint I[6];
    EllipticCurvePoint A[6];
    EllipticCurvePoint B[6];
    EllipticCurvePoint Q[6];
    EllipticCurveScalar z[6];
    EllipticCurveScalar za[6];
    EllipticCurveScalar zb[6];
    EllipticCurveScalar f;
}

TransactionKernel {
    EllipticCurvePoint excessCommitment;
    EllipticCurveScalar sigE;
    EllipticCurveScalar sigS;
}
```

```
}
```

11.3 Signing hash

```
tx_signing_hash = getObjectHash(TransactionPrefix)
```

The GK Fiat-Shamir challenge, MLSAG round hashes, and kernel signature all bind to `tx_signing_hash`. Body modification changes the full transaction hash and cannot produce an alternate valid body without the witnesses.

12. Transaction validation procedure

A CT transaction is accepted only if all checks pass in order. Implementations should fail fast and return precise diagnostic reasons in debug logs.

- Require `tx.version == TRANSACTION_VERSION_CT` and `height >= CT_FORK_HEIGHT`.
- Require `tx.unlockTime == 0`, legacy signatures empty, and outputs non-empty.
- Check `CT_MAX_INPUTS`, `CT_MAX_OUTPUTS`, and serialized transaction size limit.
- Require `ctSignatures.size() == inputs.size()` and `ctProofs.size() == outputs.size()`.
- Validate each ConfidentialInput: non-empty ring, field sizes, non-zero ringAmount, permitted ring size.
- Validate each ConfidentialOutput commitment and targetKey point encoding and subgroup membership.
- Verify each GK proof using `tx_signing_hash` and the normative transcript order from §7.6.
- Resolve each ring member by ringAmount and ring offsets; cross-check referenced public keys and commitments.
- Apply transparent member rules and V5+ coinbase ringSize==1 carve-out.
- Check spend-time unlock for every referenced ring member.
- Verify MLSAG for every input using `tx_signing_hash`.
- Check key-image uniqueness globally and within the transaction.
- Verify balance equation and kernel Schnorr signature.
- Apply fee policy.

13. Fee policy

```
CT_MINIMUM_FEE = 1010 atomic units = 0.01 KRB  
CT_MAXIMUM_FEE = 1014 atomic units = 100 KRB
```

The consensus model intentionally remains simple: CT transactions have a fixed minimum base fee, a maximum fee guard, and an additional surcharge only for oversized tx.extra data.

```
require tx.fee >= CT_MINIMUM_FEE  
require tx.fee <= CT_MAXIMUM_FEE  
required = CT_MINIMUM_FEE + getFeePerByte(tx.extra.size(), CT_MINIMUM_FEE)  
require tx.fee >= required
```

Layer	Policy
Consensus	Simple base fee, max fee, tx.extra anti-padding surcharge, size caps.
Wallet	Base / priority / high-priority fee UI; warn for large many-input transactions.
Miner	Prefer transactions by fee density or total reward contribution.
Relay policy	May locally de-prioritize very large low-fee consolidation transactions.

14. Wallet construction policy

14.1 Build sequence

```
buildConfidentialTransaction:
  1. Validate destination amounts (non-zero multiples of MIN_CT_DENOMINATION).
  2. Select inputs using canonical-first policy.
  3. Populate input vector fully; compute inputs_hash.
  4. Derive tx_key = Hs( a_s || inputs_hash ).
  5. Derive output shared secrets, blindings, commitments, target keys, masked
  amounts.
  6. Build TransactionPrefix and compute tx_signing_hash.
  7. Generate GK proofs for all CT outputs.
  8. Generate MLSAG signatures for all inputs.
  9. Compute excess and sign kernel.
  10. Securely wipe blindings, excess, and temporary secrets.
```

14.2 Input selection

Phase	Selection rule
1. Canonical/mixable	Select CT outputs and transparent outputs whose amounts are multiples of MIN_CT_DENOMINATION.
2. Non-canonical fallback	Only if needed, select sub-floor residue or non-canonical V5+ coinbase outputs largest-first.
Dust sweep command	Batch consolidation should be explicit, not automatic opportunistic cleanup.

14.3 Ring and fee UI

- Normal sends should default to ring size 16.
- The wallet should not expose ring size as a casual user preference.
- The wallet should expose fee priority, especially for many-input transactions.
- If a transaction is large, explain that a higher fee may confirm faster because miners prefer more smaller fee-paying transactions.

15. Coinbase treatment

Coinbase remains transparent at all heights. The block reward and collected fees are public consensus values, so confidential coinbase would not provide meaningful amount privacy.

Coinbase era	Output behavior	Notes
V1–V4	Legacy decomposed coinbase outputs.	
V5+	Single output of the full reward.	ringSize=1 carve-out applies at spend time.
V6+ CT era	Coinbase remains version 1 and transparent; non-coinbase transfers are CT.	

15.1 V5+ coinbase carve-out

```
ringSize == 1 is permitted only if:  
- the sole ring member is a transparent KeyOutput,  
- it is in txSlot == 0 of a block,  
- the block has majorVersion >= V5.
```

This carve-out is necessary because V5+ single-output coinbase amounts are generally not canonical denomination multiples and may have no decoys in their amount bucket. Miner privacy improves after converting coinbase into normal CT outputs and later spending from the CT pool.

16. Activation and compatibility

```
check_tx_syntax(tx, height):
    require tx.version in {1, 2}
    if tx.version == TRANSACTION_VERSION_CT and height < CT_FORK_HEIGHT:
        reject "CT before activation"
    if tx.version == 1 and getBlockMajorVersion(height) >= V6 and tx is not
    coinbase:
        reject "legacy tx in CT-only era"
```

Mempool admission should run the full validation pipeline. Reorg replay must revalidate the full CT transaction body, including GK proofs, MLSAG signatures, key images, balance equation, and kernel signature.

17. Security properties and known limitations

Property	Mechanism
Exact amount hiding	Pedersen commitments and ECDH-masked amount fields.
Denomination validity	GK one-out-of-many proof over the 64-entry table.
Inflation resistance	Balance equation plus kernel Schnorr signature.
Sender privacy	MLSAG rings; official wallet default ring size 16.
Recipient privacy	Standard CryptoNote stealth addresses.
Double-spend prevention	Global key-image uniqueness.
Audit mode	View-key-reconstructible tx_key for outgoing visibility.
No trusted setup	Discrete-log based proofs only.

17.1 Known limitations

- Output count correlates with approximate payment magnitude because amounts decompose into canonical denominations.
- Repeated 100,000 KRB cap outputs leak lower-bound magnitude information for very large transfers.
- Change-output analysis is mitigated but not eliminated by wallet policy.
- Coinbase first-spend conversion can be identifiable because of the ringSize==1 carve-out.
- The fixed base-fee model is user-friendly, but local relay/miner policy should protect against oversized low-fee spam.

18. Transaction size estimates

Component	Approx. size
GK proof	1,376 B
Confidential output total	~1,448 B (targetKey + commitment + maskedAmount + GK proof)
Ring-4 input	~636 B
Ring-8 input	~1,180 B
Ring-16 input	~2,232 B
Kernel	~96 B (excessCommitment + sigE + sigS)

A typical 1-in / 2-out CT transaction with ring size 16 occupies approximately 5,200 bytes. Wallets should warn users when a transaction exceeds 10 kB and encourage a higher priority fee.

19. Implementation plan

19.1 Consensus changes

- Add TRANSACTION_VERSION_CT = 2 and MIN_CT_DENOMINATION = 10^{10} .
- Add the 64-entry denomination table as a compile-time constant.
- Implement Pedersen commitments and canonical H generation.
- Implement standard GK one-out-of-many proof with Q[0..5] handling as specified in §7.
- Implement ECDH amount masking/unmasking with domain-separated r_out derivation.
- Implement two-row MLSAG with domain-separated round hash.
- Implement balance equation and kernel signing/verification.
- Apply strict prime-order subgroup checks at all CT validation sites.
- Add full CT transaction validation pipeline (§12).
- Add V5+ coinbase carve-out (§15.1).
- Implement prefix/body serialization conditional on transaction version.

19.2 Wallet changes

- Validate destination amounts as non-zero multiples of MIN_CT_DENOMINATION.
- Use greedy canonical decomposition in official wallets.
- Use ring size 16 by default for normal sends.
- Keep tx_key = Hs(a_s || inputs_hash) for compatibility.
- Add outgoing self-detection through deterministic tx_key regeneration.
- Implement explicit dust_sweep command for non-canonical residue.
- Expose fee priority, not casual ring-size selection.

19.3 RPC and explorer changes

- Return ring members for transparent buckets and the synthetic confidential output bucket.
- Render CT fields: commitments, masked amount, GK proof, MLSAG signatures, pseudoCommitments, and kernel.
- Report tx.fee directly for CT transactions; do not infer fee from visible input/output differences.
- Display privacy warnings for ringSize==1 coinbase conversions and unusual low-ring transactions.

20. Testing priorities

20.1 GK proof tests

- Prove and verify every denomination index 0..63.
- Verify the cap denomination at index 63.
- Reject C values that do not correspond to any denomination in $V[0..63]$.
- Reject proof generated for one C but attached to another C.
- Reject wrong tx_signing_hash.
- Reject changed denomination table order.
- Reject tampered I, A, B, Q, z, za, zb, or f.
- Reject non-canonical scalar encodings (value $\geq q$).
- Reject identity, low-order, torsion-tainted, or non-subgroup points.
- Reject the old incomplete 768-byte proof form (missing $Q[0..5]$).
- Verify independent reference implementation matches consensus implementation test vectors.

20.2 MLSAG, balance, wallet, and activation tests

- MLSAG round-trip for ring sizes 1, 4, 8, 16.
- Reject decoy tampering at every ring index.
- Reject reused key images globally and within the same transaction.
- Accept honest balanced transactions and reject imbalanced transactions.
- Reject wrong fee in balance equation.
- Reject tampered kernel signature.
- Recipient scanning identifies owned outputs and skips non-owned outputs.
- Outgoing self-detection works through deterministic tx_key regeneration.
- Sub-floor destination amount is rejected up front.
- Sub-floor change residue is folded into fee.
- CT before activation is rejected; legacy non-coinbase after activation is rejected.
- Coinbase remains version 1 and transparent after activation.

References

Groth, J., and Kohlweiss, M. "One-out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin." EUROCRYPT 2015.

Karbowanec dev/ct_20 implementation: src/crypto/gk_proof.h, src/crypto/gk_proof.cpp.

CryptoNote Whitepaper, van Saberhagen, 2013.